
ARQUITECTURAS DE REDES NEURONALES PARA INTERFACES CEREBRO-COMPUTADORA: CLASIFICACIÓN DE SEÑALES P300

Arana Chavez Emiliano, Vazquez Perez Angel Martin, Vences Guevara Alexis
Universidad del Pais Vasco
Facultad de informática
San Sebastian, Guipúzcoa

ABSTRACT

Las Interfaces Cerebro-Computadora (BCI) representan un área de investigación muy importante cuyo objetivo es permitir la comunicación entre el cerebro humano y sistemas mediante el análisis de señales neuronales. Dentro de este campo, el p300 speller es un sistema que se basa en la idea de que, cuando una persona presta atención a un estímulo relevante, su cerebro genera una respuesta eléctrica 300 milisegundos después del estímulo. Esta respuesta es la llamada señal P300.

En este proyecto se presenta el diseño, implementación y evaluación de dos arquitecturas de redes neuronales profundas para la detección de señales P300 a partir de señales electroencefalográficas (EEG). Se emplea el Dataset II de la competencia BCI III, utilizando dos modelos principales: una red neuronal convolucional (CNN) convencional y la arquitectura EEGNet, diseñada específicamente para el procesamiento de señales EEG. así también un modelo híbrido que implementa Redes Recurrentes. El objetivo principal de los modelos propuestos no es predecir directamente el carácter observado por el sujeto, sino clasificar ventanas temporales de EEG en dos clases: presencia o ausencia de la señal P300. Posteriormente, las predicciones se agrupan por filas y columnas del *P300 Speller* para realizar la decodificación final del carácter.

Este enfoque representa una evolución con respecto a los métodos clásicos utilizados en las primeras competencias de Interfaces Cerebro-Computadora, donde la detección de la señal P300 se realizaba mediante clasificadores lineales y máquinas de soporte vectorial (SVM) [1]. A diferencia de estos enfoques, los modelos basados en aprendizaje profundo permiten aprender de manera automática representaciones espaciales y temporales directamente a partir de las señales EEG, reduciendo la necesidad de ingeniería manual de características.

Keywords BCI · P300 · EEG · Deep Learning · CNN · LSTM · EEGNet

1. Descripción del problema

El problema abordado en este proyecto consiste en la detección automática de las señales P300 dentro de señales EEG altamente con mucho ruido y de baja amplitud. Como se mencionó anteriormente, la señal P300 es una respuesta cerebral que ocurre aproximadamente 300 milisegundos después de que un sujeto percibe un estímulo relevante dentro de una secuencia de estímulos irrelevantes, lo que la hace especialmente útil en paradigmas de selección como el P300 Speller Sin embargo, la detección de esta señal varios problemas.

Las señales EEG poseen una gran cantidad de ruido en su información, pues están altamente influenciadas por artefactos fisiológicos y ambientales, y presentan una gran variabilidad entre los sujetos de prueba. Además, la respuesta P300 no ocurre de manera idéntica en todas las repeticiones, lo que dificulta su identificación mediante métodos tradicionales. En el contexto del experimento P300 Speller, un sujeto fija su atención visual en un carácter dentro de una matriz de caracteres (figura 1), mientras las filas y columnas de dicha matriz se iluminan de forma secuencial y aleatoria. Cada iluminación constituye un estímulo visual que puede ser relevante o irrelevante para el usuario, dependiendo de si contiene o no el carácter seleccionado. Cuando el estímulo es relevante, el cerebro genera la señal P300 detectable en la señal EEG. De este modo, cada carácter no está asociado a una única señal, sino a una secuencia de respuestas EEG correspondientes a múltiples iluminaciones de filas y columnas. En consecuencia, el problema no se limita a

la clasificación individual de señales EEG, sino que requiere la integración de múltiples predicciones para inferir de manera más segura y precisa el carácter finalmente seleccionado por el usuario.

Ante estas dificultades, el uso de redes neuronales profundas surge como solución, capaz de aprender automáticamente la información espacial y temporal relevante de las señales a partir de los datos de EEG.

2. Descripción de los datos

Los datos utilizados provienen del Dataset II de la competencia BCI III (<https://www.bbcii.de/competition/iii/>). Contiene registros de dos sujetos (A y B) con 64 canales de EEG, muestreados originalmente a 240 Hz. Para cada sujeto se dispone de 85 caracteres para entrenamiento y 100 para prueba.



Figura 1: Matriz del P300 Speller

La matriz utilizada durante los experimentos consistió en una matriz de 6×6 caracteres. Cada época de un carácter incluye:

- 12 intensificaciones (6 filas y 6 columnas) aleatorias.
- Duración de intensificación: 100 ms.
- Intervalo entre intensificaciones: 75 ms.
- 15 repeticiones por carácter, resultando en 180 intensificaciones totales por carácter.

Las variables principales del dataset son:

- **Signal:** es una matriz que contiene la señal EEG registrada por los 64 electrodos muestreada a 240 Hz. Los canales se encuentran ordenados según su numeración mostrada en la figura 2
- **Flashing:** variable binaria que indica si una fila o columna está siendo intensificada (1: intensificada, 0: no intensificada)
- **StimulusCode:** Indica la fila/columna intensificada (1-6 columnas, 7-12 filas, 0 sin intensificación).3
- **StimulusType:** Etiqueta binaria para detección (1: contiene carácter objetivo/P300, 0: no contiene).
- **TargetChar:** Carácter objetivo (solo en entrenamiento).

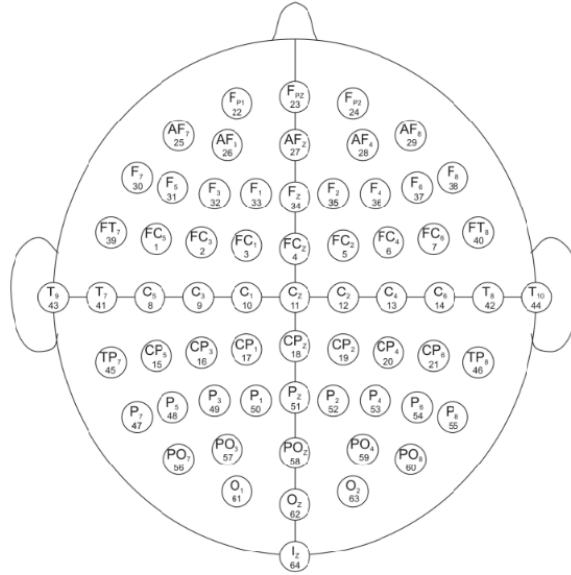


Figura 2: Distribucion de electrodos en la cabeza.



Figura 3: Distribucion de electrodos en la cabeza.

2.1. Visualizacion de datos

Podemos hacer una pequeña visualizacion de como es que estan organizados nuestros datos. LA 4 nos muestra todos los datos que previamente mencionamos, de color amarillo tenemos el flashing, de color azul podemos visualizar la señal EEG de nuestro electrodo seleccionado, vemos el valor binario de StimulusTyp e indicandonos que en esa fila y columan que code nos indica, es el caracter objetivo que el usuario esta pensando.

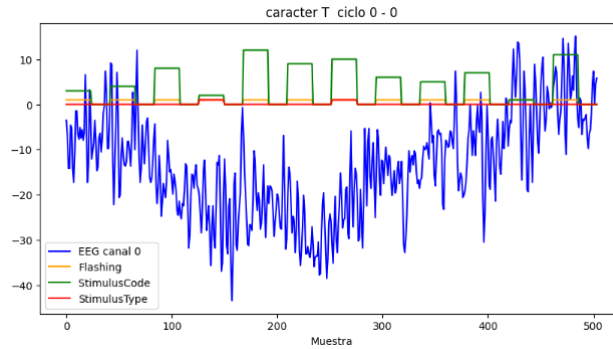


Figura 4: Visualización de datos disponibles (Signal: Azul, StimulusCode: Verde, Flashing: Amarillo, StimulusType: Rojo.)

3. Preprocesado de los Datos

Una parte fundamental de este trabajo es el preprocesado de los datos, ya que no es conveniente ingresar las señales EEG en crudo directamente a los modelos de clasificación, ya que se dificultaría el proceso de aprendizaje y detección de la señal P300. Por lo tanto, antes de realizar la segmentación en ventanas y el entrenamiento de los modelos, se aplicó una serie de pasos de preprocesado con el objetivo de mejorar la calidad de la señal, reducir ruido innecesario y normalizar los datos para que todos los canales contribuyan de manera balanceada al proceso de clasificación. Se aplicaron las siguientes etapas:

3.1. Procesamiento de señal EEG

1. **Filtro Paso Banda:** Se aplicó un filtro entre 0.1 Hz y 20 Hz. Este rango conserva la información del P300 y elimina derivas de baja frecuencia y ruido muscular/eléctrico de alta frecuencia.
2. **Remuestreo:** Se redujo la frecuencia de 240 Hz a 120 Hz para disminuir la carga computacional sin perder información temporal relevante.
3. **Normalización Z-score:** Posteriormente, se aplicó una normalización tipo Z-score de manera independiente para cada canal EEG. Este proceso consiste en restar la media y dividir entre la desviación estándar de cada canal. La normalización permite: Evitar que canales con mayor amplitud dominen el proceso de aprendizaje. Mejorar la estabilidad del entrenamiento de las redes neuronales. Facilitar la comparación entre señales provenientes de distintos electrodos.

3.2. Procesado de señales discretas

Además de la señal EEG continua, el dataset incluye señales discretas como Flashing, StimulusCode y StimulusType, las cuales indican la ocurrencia de estímulos visuales y su relevancia. Dado que la señal EEG fue remuestreada de 240 Hz a 120 Hz, fue necesario aplicar el mismo proceso de remuestreo a estas señales discretas para asegurar que todas las variables permanecieran alineadas temporalmente y compartieran la misma frecuencia de muestreo. Este paso es fundamental para garantizar que cada muestra de EEG corresponda correctamente al estímulo visual asociado.

3.3. Segmentación de ventanas

Las señales continuas y las variables discretas (*Flashing*, *StimulusCode*) Una vez preprocesadas y alineadas todas las señales, se procedió a la segmentación de la señal EEG en ventanas temporales, las cuales constituyen las muestras de entrada para los modelos de clasificación. Para el análisis de la señal P300, se decidió utilizar ventanas de 78 muestras, lo que corresponde a una duración aproximada de 0.65 segundos, considerando una frecuencia de muestreo de 120 Hz. Este intervalo temporal es adecuado para capturar la respuesta P300 completa posterior a un estímulo visual. Las ventanas se generan alineadas al inicio de cada intensificación (flash), es decir, cada vez que ocurre un estímulo visual relevante o irrelevante, se extrae una ventana de EEG a partir de ese instante. Cada ventana se asocia con una etiqueta binaria obtenida de la variable StimulusType, donde:

- **Clase 1 (P300):** Estímulo relevante.
- **Clase 0 (No-P300):** Estímulo irrelevante.

La dimensión final de los datos de entrada es: (Total de ventanas, 1, 64 canales, 78 muestras).

4. Diseño e implementación de Arquitecturas

Para poder clasificar este tipo de información investigamos varias fuentes y las soluciones más óptimas era implementar arquitecturas basadas en redes neuronales convolucionales ya que estas pueden tomar estas características de la señal tanto espacial como temporal si se manejan muy bien, A continuación se hará una descripción de las arquitecturas utilizadas y como fue la implementación en python

4.1. CNN clásica para P300

Basada en el trabajo de Cecotti y Gräser [2]. Se implementó una red neuronal convolucional, que fue diseñada específicamente para la detección de la señal P300 en señales EEG. La red está compuesta por cinco capas, tal como se muestra en la (figura 5), considerando la capa de entrada. La arquitectura se encuentra organizada en distintos mapas de características, separando explícitamente el dominio espacial y el dominio temporal, lo que facilita tanto el aprendizaje como la interpretación de los filtros obtenidos. A continuación, se describe cada una de las capas que conforman el modelo.

- **Entrada:** La capa de entrada recibe ventanas de señales EEG de tamaño 64×78 , donde 64 corresponde al número de electrodos utilizados y 78 al número de muestras temporales por ventana. Estas ventanas representan segmentos de la señal EEG alineados temporalmente con los estímulos visuales del experimento P300 Speller.
- **Capa L1 (Espacial):** La primera capa convolucional tiene como objetivo realizar un análisis espacial de la señal EEG. Para ello, se utiliza una convolución 2D. Esta operación permite colapsar la dimensión espacial, aprendiendo combinaciones lineales entre los electrodos que resultan relevantes para la detección de la señal P300. Como salida se obtienen N_s mapas de características, cada uno representando una posible distribución espacial del potencial P300.
- **Capa L2 (Temporal):** La segunda capa convolucional está orientada al análisis temporal de la señal. En esta etapa se aplican convoluciones con filtros de tamaño (1×13) , los cuales se desplazan únicamente en la dimensión temporal: Esta capa permite capturar patrones temporales característicos de la señal P300, como cambios de amplitud y forma de onda que ocurren alrededor de los 300 ms posteriores al estímulo. Además, debido al tamaño del kernel, la longitud temporal se reduce de 78 a 66 muestras, realizando implícitamente un submuestreo temporal. Al igual que en la capa anterior, se utiliza la función de activación tanh.
- **Capa L3 (Fully Connected):** Las características extraídas por las capas convolucionales se aplanan para formar un vector de características. Este vector es procesado por una capa totalmente conectada de 100 neuronas. Esta capa se encarga de combinar las características espaciales y temporales aprendidas previamente, generando una representación de alto nivel de cada ventana EEG. Se utiliza la función de activación Sigmoide.
- **Capa L4 (Salida):** Finalmente, la capa de salida está compuesta por dos neuronas, las cuales representan las dos clases del problema: (P300 o no) La salida de esta capa se utiliza directamente junto con la función de pérdida de entropía cruzada, por lo que no es necesario definir explícitamente una función Softmax en el modelo. La representación de la arquitectura se muestra en la 5

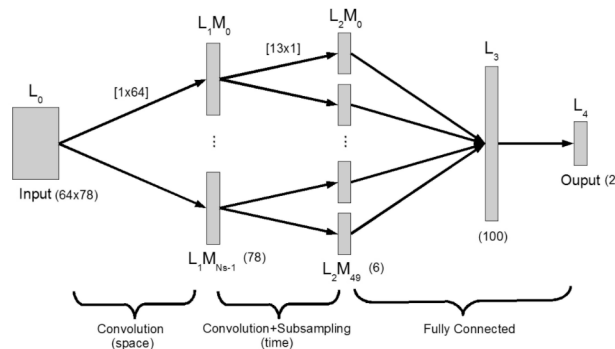


Figura 5: Representación de la arquitectura CNN clásica para P300.

4.2. EEGNet

4.3. Arquitectura EEGNet

Como segundo experimento se implementó la arquitectura **EEGNet**, una red neuronal convolucional diseñada específicamente para el análisis de señales EEG dentro de sistemas de Brain–Computer Interface (BCI). A diferencia de las arquitecturas convolucionales convencionales, EEGNet fue propuesta para adaptarse a las características intrínsecas de las señales EEG, tales como la baja relación señal-ruido y la limitada cantidad de datos disponibles en la mayoría de los experimentos BCI. La arquitectura implementada se describe detalladamente a continuación??..

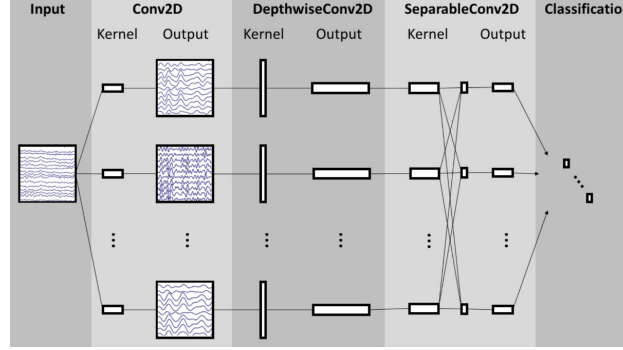


Figura 6: Representación de la arquitectura EEGNet.

4.3.1. Bloque 1: Convolución temporal y filtrado espacial (Conv2D + Depthwise)

El primer bloque de la arquitectura corresponde a las capas encargadas de extraer características básicas de la señal, separando el procesamiento temporal del espacial. La entrada al modelo tiene una forma $[batch, 64, 78]$, donde 64 representa los electrodos y 78 las muestras temporales. En el método *forward*, esta entrada se reorganiza mediante $x = x.unsqueeze(1)$, quedando con forma $[batch, 1, 64, 78]$, lo que permite aplicar convoluciones bidimensionales.

Convolución temporal La primera parte del bloque es una convolución 2D temporal, definida como:

```
self.conv1 = nn.Conv2d(
    in_channels=1,
    out_channels=F1,
    kernel_size=(1, 16),
    padding=(0, 8),
    bias=False
)
```

Esta capa aplica $F1$ (8) filtros de tamaño (1×16) , lo que significa que cada filtro se desplaza únicamente sobre la dimensión temporal, sin combinar información entre canales aún. En esta etapa se aprenden patrones temporales de la señal EEG, como cambios de amplitud y formas de onda relacionados a la señal P300. El uso de *padding* permite conservar la longitud temporal de la señal, asegurando que las dimensiones no se alteren drásticamente.

Convolución espacial Depthwise Una vez extraídas las características temporales, se aplica una convolución *depthwise* espacial:

```
self.depthwise = nn.Conv2d(
    in_channels=F1,
    out_channels=F1 * D,
    kernel_size=(Nelec, 1),
    groups=F1,
    bias=False
)
```

Esta capa utiliza un *kernel* de tamaño (64×1) que abarca todos los canales simultáneamente, pero mantiene separadas las características temporales aprendidas anteriormente. El parámetro `groups=F1` fuerza a que cada filtro temporal

tenga su propio conjunto de filtros espaciales. De esta manera, el modelo aprende cómo se distribuye espacialmente la señal P300, permitiendo capturar patrones característicos de distintas partes del cerebro.

Es importante mencionar el uso de **Dropout** (`self.drop1 = nn.Dropout(dropout)`) para desactivar aleatoriamente neuronas durante el entrenamiento, ayudando a prevenir el sobreajuste.

4.3.2. Bloque 2: Convolución separable (SeparableConv2D)

El segundo bloque de EEGNet se encarga de combinar las características extraídas anteriormente utilizando una convolución separable, implementada mediante dos operaciones consecutivas.

Convolución depthwise temporal Aplica un filtro temporal a cada mapa de características. Dado que la señal fue previamente reducida por un `AvgPool2d(kernel_size = (1, 4))`, la ventana temporal de `kernel_size=(1, 8)` equivale aproximadamente a 500ms de actividad EEG.

```
nn.Conv2d(
    in_channels=F1 * D,
    out_channels=F1 * D,
    kernel_size=(1, 8),
    padding=(0, 4),
    groups=F1 * D,
    bias=False
)
```

Convolución pointwise Posteriormente, se aplica una convolución *pointwise* de tamaño (1×1) :

```
nn.Conv2d(
    in_channels=F1 * D,
    out_channels=F2,
    kernel_size=1,
    bias=False
)
```

Esta capa finalmente combina la información de los distintos mapas, aprendiendo qué combinaciones de características son más importantes para la clasificación P300 vs No-P300.

4.3.3. Activación y reducción final

Después de la convolución separable, se aplica una activación **ELU** (`self.act2 = nn.ELU()`) y una segunda capa de **Average Pooling**

```
(self.pool2 = nn.AvgPool2d(kernel_size=(1, 8)))
```

para reducir la dimensión temporal. Se incluye una etapa final de **Dropout** antes de la clasificación.

4.3.4. Bloque de clasificación

Tras las etapas convolucionales, las características quedan aplanadas para ser procesadas por la capa *fully connected*. Para determinar automáticamente el tamaño de entrada de esta capa, se utiliza un tensor de prueba (*dummy*):

```
# Cálculo automático de la dimensión de entrada
with torch.no_grad():
    dummy = torch.zeros(1, 1, Nelec, Nt)
    o = self.pool1(self.act1(self.bn2(self.depthwise(self.bn1(self.conv1(dummy))))))
    o = self.pool2(self.act2(self.bn3(self.separable(o))))
    flat = o.reshape(1, -1).shape[1]

self.fc = nn.Linear(flat, num_classes)
```

El uso del *dummy* asegura que la capa `self.fc` reciba la cantidad exacta de características necesarias para producir la salida final, permitiendo que la interpretación de si una ventana contiene un potencial P300 se integre eficientemente en el proceso de entrenamiento.

4.4. Arquitectura Híbrida (CNN + LSTM)

Como tercera arquitectura se propuso una arquitectura híbrida que combina redes neuronales convolucionales (CNN) con redes neuronales recurrentes (RNN), específicamente una LSTM bidireccional, con el objetivo de cambiar la forma de trabajar la información en el tiempo de la señal EEG. A diferencia de las arquitecturas previamente mencionadas, que capturan dependencias temporales mediante filtros de tamaño fijo, el LSTM está diseñado para aprender dependencias temporales de corto y largo plazo, lo cual resulta especialmente adecuado para la detección de la señal P300, cuya aparición es temporalmente consistente pero presenta variabilidad entre repeticiones y sujetos. A continuación se presenta como es esta arquitectura

1. **CNN Espacial y Temporal:** Filtros iniciales similares a la CNN clásica para P300.
2. **Reshape:** Se permutan las dimensiones para tratar el tiempo como secuencia.
3. **LSTM Bidireccional:** Modela dependencias temporales de largo alcance y variabilidad de la latencia del P300.
4. **Clasificación:** Promedio temporal de la salida LSTM seguido de una capa lineal.

5. Decodificación de caracteres

Como se mencionó anteriormente los modelos desarrollados, no predicen el carácter final que el sujeto desea seleccionar directamente, si no que hacen la clasificación binaria indicando únicamente si una ventana contiene o no una señal P300. Por ello es necesario incluir un módulo para manejar la decodificación del dígito en el que tentativamente estaría fijando como objetivo el sujeto de prueba al momento de la señal P300. Para ello, el primer paso consiste en evaluar el modelo sobre el conjunto de prueba y obtener para cada ventana EEG la predicción binaria y la probabilidad correspondiente a la clase P300. Esto se hace mediante la función `testp300`:

```
out = model(X) prob = torch.softmax(out, dim=1)[:, 1] pred = torch.argmax(out, dim=1)
```

Aquí, la salida del modelo corresponde a dos valores por ventana EEG. A partir de estos valores, se aplica Softmax para obtener la probabilidad de la clase P300, mientras que la predicción binaria se obtiene mediante `argmax`. Además, se conservan los códigos de estímulo (`stimulusCode`), que indican si la ventana corresponde a una fila o a una columna de la matriz del speller, y el carácter real asociado a esos estímulos. Así, el resultado de la función es un diccionario con la predicción binaria, probabilidades asociadas a la señal P300, Códigos de estímulo (fila y columna en la tabla), y las etiquetas reales. Por otro lado, para la propia decodificación del carácter se usa la acumulación de probabilidades P300:

$$Score_{fila_i} = \sum P(x|x \in fila_i) \quad (1)$$

$$Score_{col_j} = \sum P(x|x \in col_j) \quad (2)$$

El carácter predicho es la intersección de la fila y columna con los puntajes máximos acumulados tras las 15 repeticiones.

6. Análisis de resultados

En esta sección se presentan y analizan los resultados obtenidos en las pruebas a las 3 arquitecturas implementadas. Para cada arquitectura se probaron distintas configuraciones de hiperparámetros para buscar mejorar la precisión en la identificación de caracteres. Adicionalmente, se realizaron pruebas utilizando subconjuntos de electrodos seleccionados por su relevancia en la señal P300, con el fin de evaluar el desempeño de los modelos con menos datos y ver si el desempeño mejora o por el contrario empeora respecto al obtenido con todo el conjunto de canales. En primer lugar, se definieron diferentes configuraciones de hiperparámetros para entrenar los modelos y comparar el rendimiento obtenido entre las diferentes configuraciones.

6.1. Configuración Experimental

Para cada arquitectura se probaron distintas configuraciones de hiperparámetros para buscar mejorar la precisión en la identificación de caracteres. Adicionalmente, se realizaron pruebas utilizando subconjuntos de electrodos seleccionados por su relevancia en la señal P300. A continuación, se detallan las configuraciones utilizadas.

Tabla 1: Configuración de hiperparámetros para la Red Convolutiva clásica para P300 (CNN)

Exp.	Channels	Ns	LR	WD	Epochs	Batch Size
1	all	10	0.0001	0.00001	30	128
2	all	10	0.00005	0.00001	60	128
3	all	16	0.0001	0.00001	30	256
4	15	10	0.0001	0.00001	40	128
5	10	10	0.0001	0.00001	30	128

Tabla 2: Configuración de experimentos para la arquitectura EEGNet

Exp.	Channels	LR	WD	Epochs	Batch Size
1	all	0.0001	0.00001	300	128
2	all	0.0001	0.00005	300	256
3	15	0.0001	0.00005	300	128
4	10	0.0001	0.00005	300	128

Tabla 3: Configuración de experimentos para la arquitectura Híbrida (RNN)

Exp.	Channels	Ns	Hidden	LR	WD	Epochs	Batch Size
1	all	10	128	0.0001	0.00001	80	128

6.2. Métricas de Evaluación

Para evaluar el desempeño de los modelos se emplearon las métricas de Accuracy, Precision, Recall, F1-score y Accuracy de Letras. Estas métricas permiten analizar el comportamiento de los clasificadores desde distintas perspectivas.

La **Accuracy** (exactitud) representa la proporción de predicciones correctas realizadas por el modelo respecto al total de muestras evaluadas:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

Donde:

- **TP (True Positives):** estímulos objetivo correctamente clasificados.
- **TN (True Negatives):** estímulos no objetivo correctamente clasificados.
- **FP (False Positives):** estímulos no objetivo clasificados incorrectamente como objetivo.
- **FN (False Negatives):** estímulos objetivo clasificados incorrectamente como no objetivo.

La **Precision** mide la proporción de predicciones positivas que realmente son acertadas:

$$Precision = \frac{TP}{TP + FP} \quad (4)$$

El **Recall** cuantifica la capacidad del modelo para detectar correctamente los estímulos objetivo:

$$Recall = \frac{TP}{TP + FN} \quad (5)$$

El **F1-score** corresponde a la media armónica entre Precision y Recall, proporcionando una evaluación balanceada del desempeño del modelo:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (6)$$

Finalmente, el **Accuracy de Letras** mide el porcentaje de caracteres correctamente identificados a partir de la salida del clasificador, considerando el proceso completo de decodificación. Esta métrica es la más importante, pues evalúa el desempeño del modelo a nivel funcional, ya que el objetivo final del sistema P300 Speller es la identificación de caracteres y no solo la clasificación de ventanas aisladas.

6.3. Resultados experimentales

Con el fin de analizar el comportamiento del entrenamiento, en la Figura 7 se muestra la evolución de la función de pérdida para el conjunto de entrenamiento y validación en la arquitectura CNN. La gráfica permite observar la convergencia del modelo y evaluar el proceso de aprendizaje.

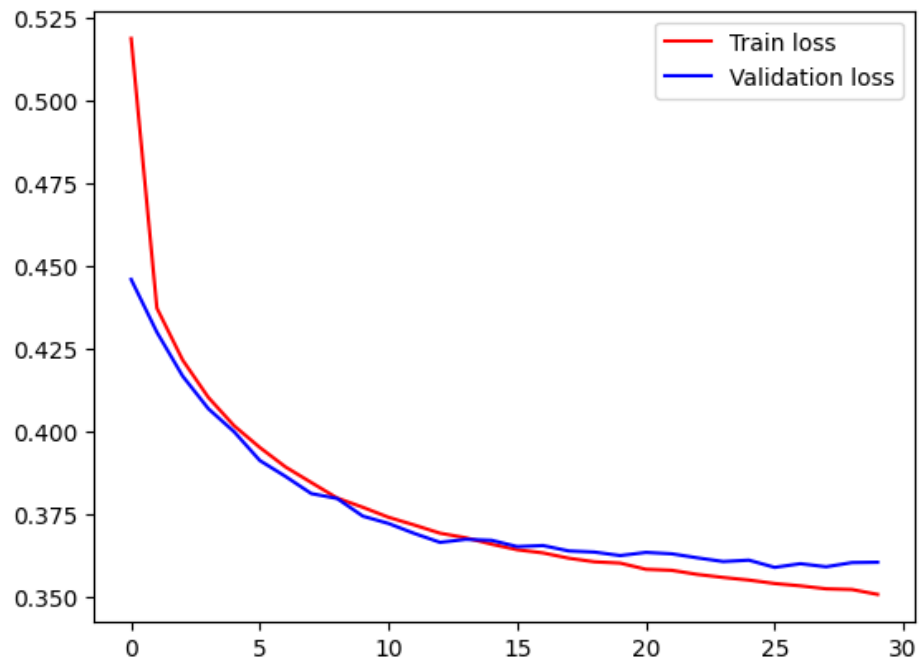


Figura 7: Evolución del Training Loss y Validation Loss para la arquitectura CNN clásica.

Las métricas obtenidas para los experimentos listados se muestran en las siguientes tablas. En este caso, la columna *Accuracy Letras* mide la precisión con la que se obtuvieron las letras adecuadas una vez identificadas las señales P300, lo cual representa el objetivo funcional de este ejercicio.

Tabla 4: Métricas obtenidas para la CNN clásica para P300

Exp.	Sujeto	Accuracy	Precision	Recall	F1-score	Acc. Letras
1	A	0.8434	0.5676	0.2547	0.3516	0.96
	B	0.8586	0.6401	0.3457	0.4489	0.95
2	A	0.8432	0.5638	0.2623	0.3581	0.94
	B	0.8589	0.6486	0.3347	0.4415	0.95
3	A	0.8422	0.5544	0.2717	0.3547	0.93
	B	0.8573	0.6230	0.3647	0.4601	0.96
4	A	0.8327	0.4553	0.0187	0.0359	0.63
	B	0.8333	0.5000	0.0913	0.1545	0.73
5	A	0.8330	0.4545	0.0100	0.0196	0.69
	B	0.8324	0.4298	0.0173	0.0330	0.38

Tabla 5: Métricas obtenidas para EEGNet

Exp.	Sujeto	Accuracy	Precision	Recall	F1-score	Acc. Letras
1	A	0.8407	0.5787	0.1630	0.2544	0.89
	B	0.8599	0.6630	0.3247	0.4359	0.98
2	A	0.8414	0.5849	0.1677	0.2606	0.85
	B	0.8557	0.6351	0.3157	0.4217	0.98
3	A	0.8336	0.8571	0.0020	0.0040	0.72
	B	0.8338	0.5247	0.0283	0.0538	0.76
4	A	0.8335	1.0000	0.0010	0.0020	0.69
	B	0.8333	0.5000	0.0013	0.0027	0.45

Tabla 6: Métricas obtenidas para Arquitectura Híbrida (RNN)

Exp.	Sujeto	Accuracy	Precision	Recall	F1-score	Acc. Letras
1	A	0.8322	0.4893	0.1520	0.2319	0.80
	B	0.8514	0.6385	0.2497	0.3590	0.94

Finalmente, en la Figura 8 se presenta la matriz de caracteres reales frente a caracteres predichos obtenida para la arquitectura CNN. Esta representación permite evaluar el desempeño funcional del sistema P300 Speller, mostrando la capacidad del modelo para identificar correctamente los caracteres a partir del proceso completo de detección de señales P300 y decodificación de filas y columnas.

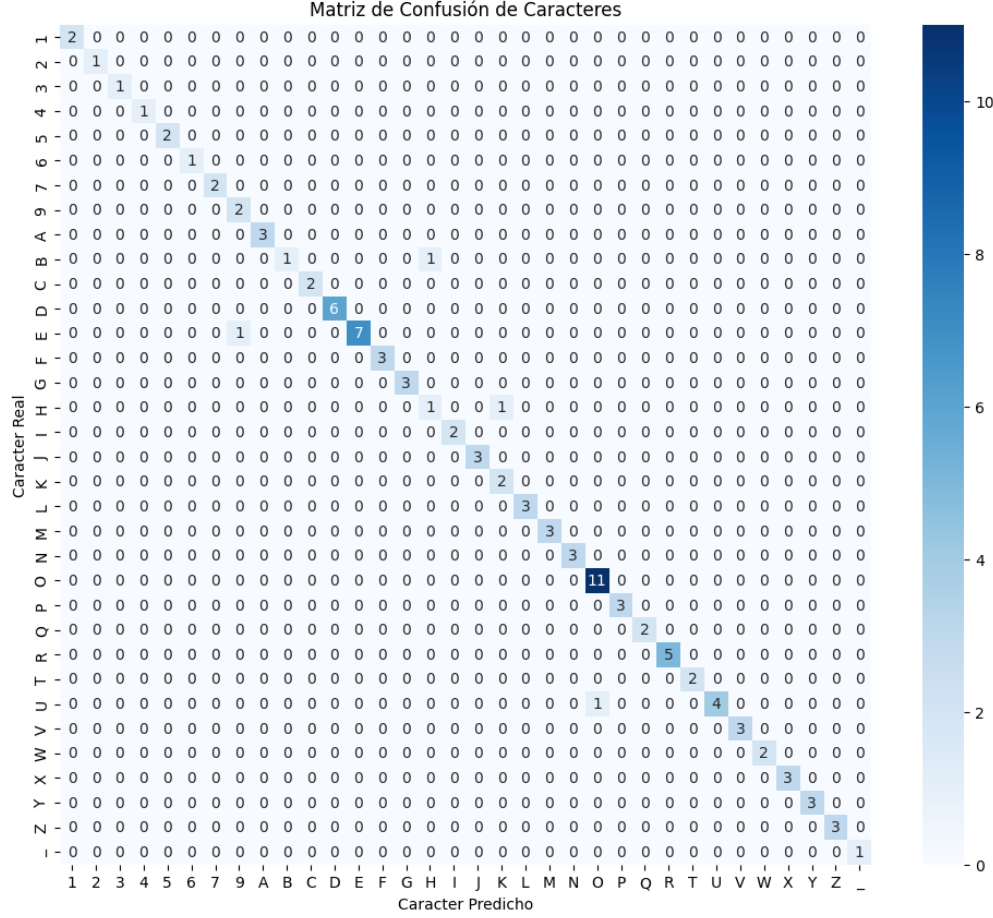


Figura 8: Matriz de caracteres reales vs caracteres predichos para la arquitectura CNN.

7. Discusión y Análisis

Como muestran los resultados obtenidos, se obtuvo un mayor rendimiento para el sujeto B con las arquitecturas más complejas, como EEGNet o el modelo CNN-RNN (Híbrido). Este comportamiento puede atribuirse a la posible variabilidad de las señales P300 entre sujetos, sumado a la mayor capacidad de estos modelos para modelar dependencias espaciales y temporales complejas.

Por otra parte, para el sujeto A, el modelo **CNN básico** presentó un desempeño claramente superior. Esto sugiere que, para el tipo de señales cerebrales de este sujeto, la arquitectura más simple resulta suficiente para capturar las características relevantes sin caer en sobreajuste. Aun así, el desempeño de este modelo básico para el sujeto B fue bastante bueno, por lo que probablemente la diferencia radica más en que los modelos complejos no lograron extraer correctamente todas las características relevantes del sujeto A debido a la naturaleza de sus datos.

Con respecto al proceso de entrenamiento, se observó que los modelos complejos necesitaron una mayor cantidad de épocas para alcanzar un comportamiento estable, mientras que el modelo básico logró converger rápidamente, obteniendo mejores resultados con un menor costo computacional.

Al analizar las métricas de desempeño, se aprecia que, en general, los modelos presentan valores bajos en Recall y F1-score, así como una precisión moderada. Este comportamiento probablemente se debe al gran desbalance entre clases, donde los estímulos "no objetivo" son considerablemente más frecuentes que los estímulos "objetivo" (relación 5:1 en el protocolo P300).

A pesar de lo anterior, la métrica de *Accuracy de Letras* alcanzó valores elevados en la mayoría de las configuraciones, lo que indica que el sistema es capaz de identificar correctamente el carácter deseado mediante la agregación de múltiples estímulos. Esto demuestra que, incluso con un bajo rendimiento en las métricas tradicionales de clasificación binaria ventana a ventana, la detección final de caracteres puede ser muy alta gracias a la repetición de intensificaciones.

Cabe destacar que las configuraciones que emplearon subconjuntos reducidos de canales (10 y 15 canales) no produjeron resultados satisfactorios para el modelo CNN-RNN, obteniendo valores de *accuracy* de letras inferiores al 30 %. Por esta razón, dichos resultados no fueron considerados relevantes para el análisis final en la Tabla 6. Sin embargo, incluso en las arquitecturas en las que se obtuvieron mejores resultados con estos canales reducidos (Tablas 4 y 5, Experimentos 4 y 5), los resultados fueron claramente inferiores a los obtenidos empleando todos los canales. De esto se puede inferir que incluso los canales que no presentan tantas variaciones visuales aportan características significativas y complementarias para la detección robusta de señales P300.

Finalmente, de todas las arquitecturas y configuraciones evaluadas, la **CNN clásica para P300 en su configuración número 1** presentó el mejor desempeño global en la detección correcta de caracteres, alcanzando un **96 %** de *accuracy* de letras para el sujeto A y un **95 %** para el sujeto B. Estos resultados, además de ser elevados, muestran un comportamiento balanceado entre sujetos, lo que resalta la efectividad de una arquitectura sencilla y eficiente frente a modelos más complejos para esta tarea específica.

8. Conclusión

En este trabajo se demostró que una mayor complejidad en la arquitectura no garantiza un mejor desempeño para todos los sujetos en sistemas BCI. La CNN clásica para P300 presentó el mejor balance entre eficiencia computacional y precisión global (superando el 95 % en ambos sujetos). EEGNet y CNN-LSTM mostraron potencial, especialmente en el sujeto B, pero requieren mayor ajuste. El sistema propuesto es viable para la decodificación precisa de caracteres a pesar del ruido inherente al EEG.

Material Suplementario

El código está disponible en Google Colab: https://colab.research.google.com/drive/13_nE6Q300th5ofqM83BH64eeUfs4fDJy

Referencias

- [1] Alain Rakotomamonjy, Vincent Guigue, Stéphane Mallet, and Pierre Alquier. Ensemble of support vector machines for p300 speller brain–computer interface. *IEEE Transactions on Biomedical Engineering*, 55(3):1147–1154, mar 2008.
- [2] H. Cecotti and A. Graeser. Convolutional neural networks for p300 detection with application to brain-computer interfaces. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(3):433–445, mar 2011.